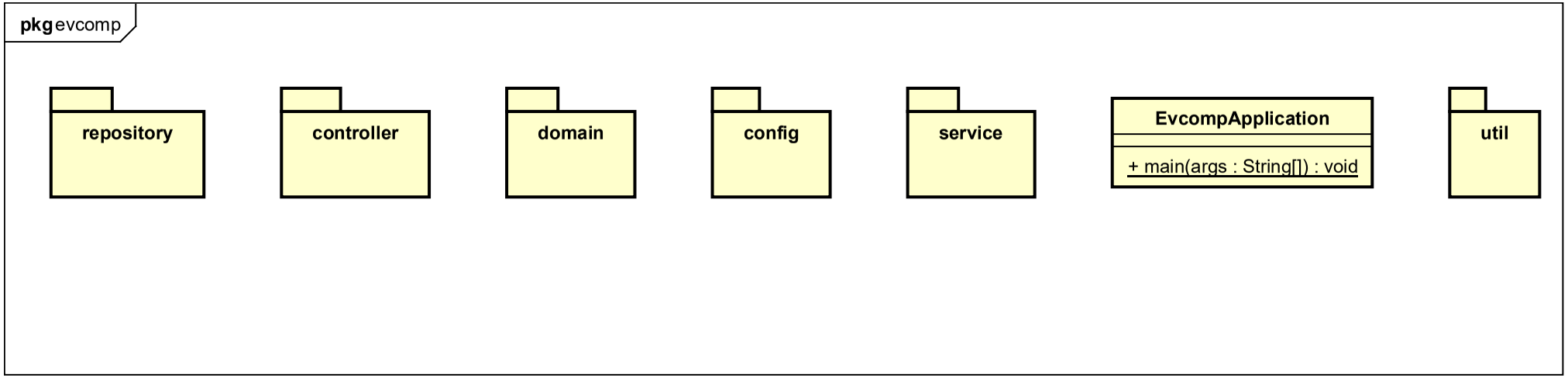
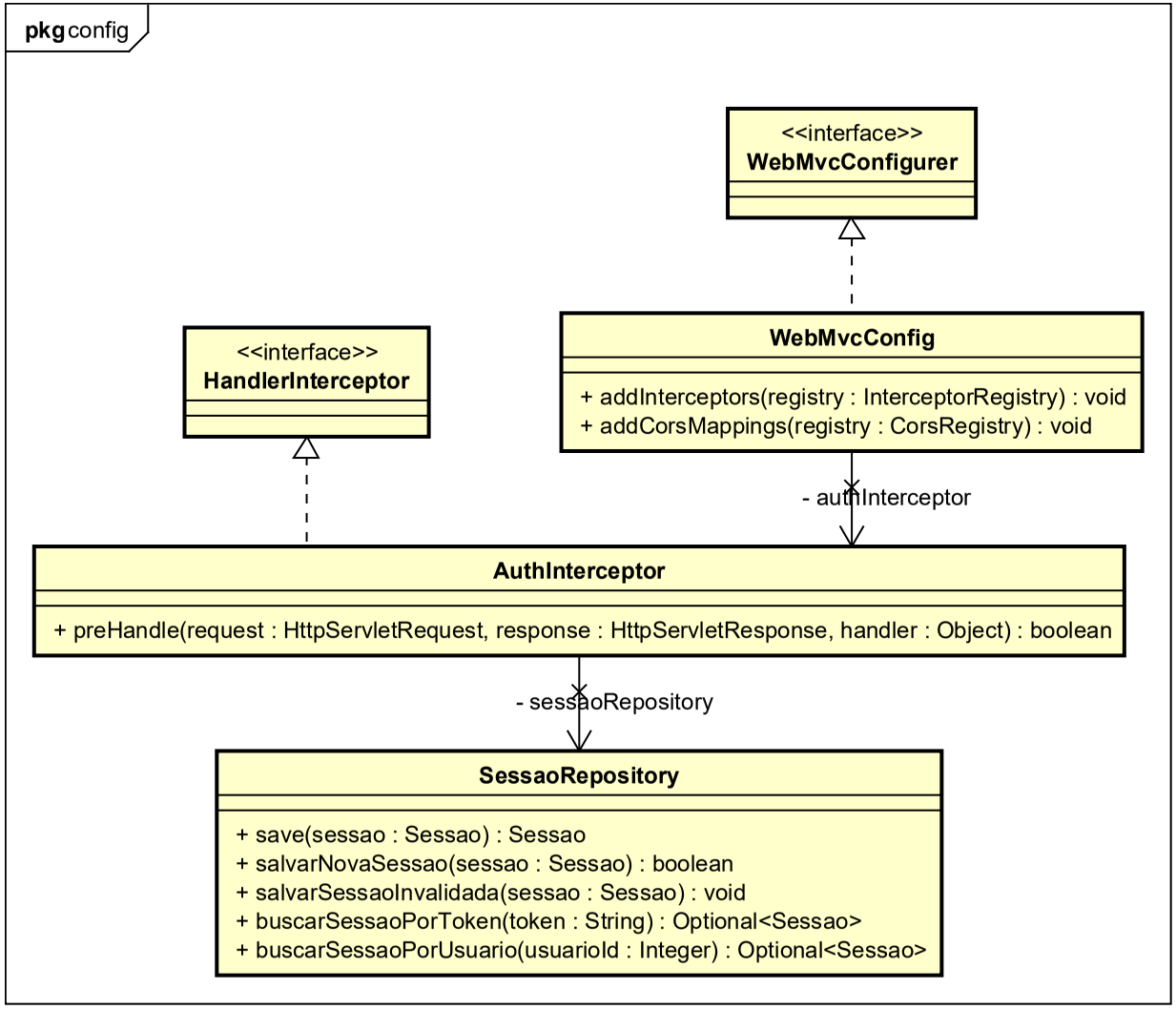
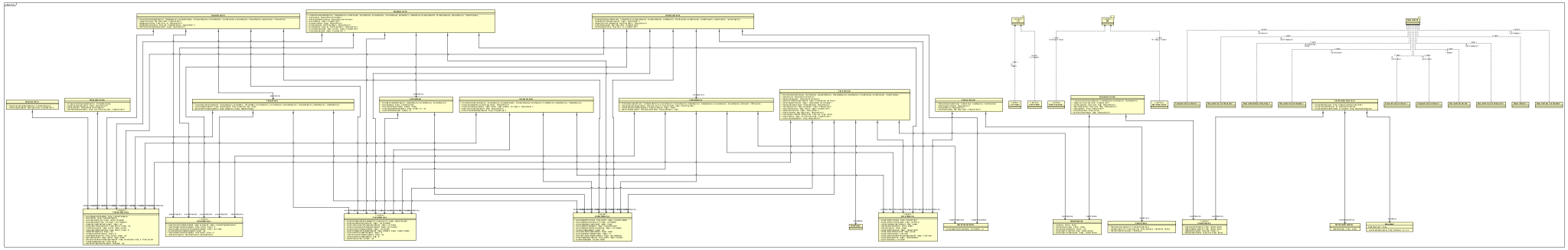
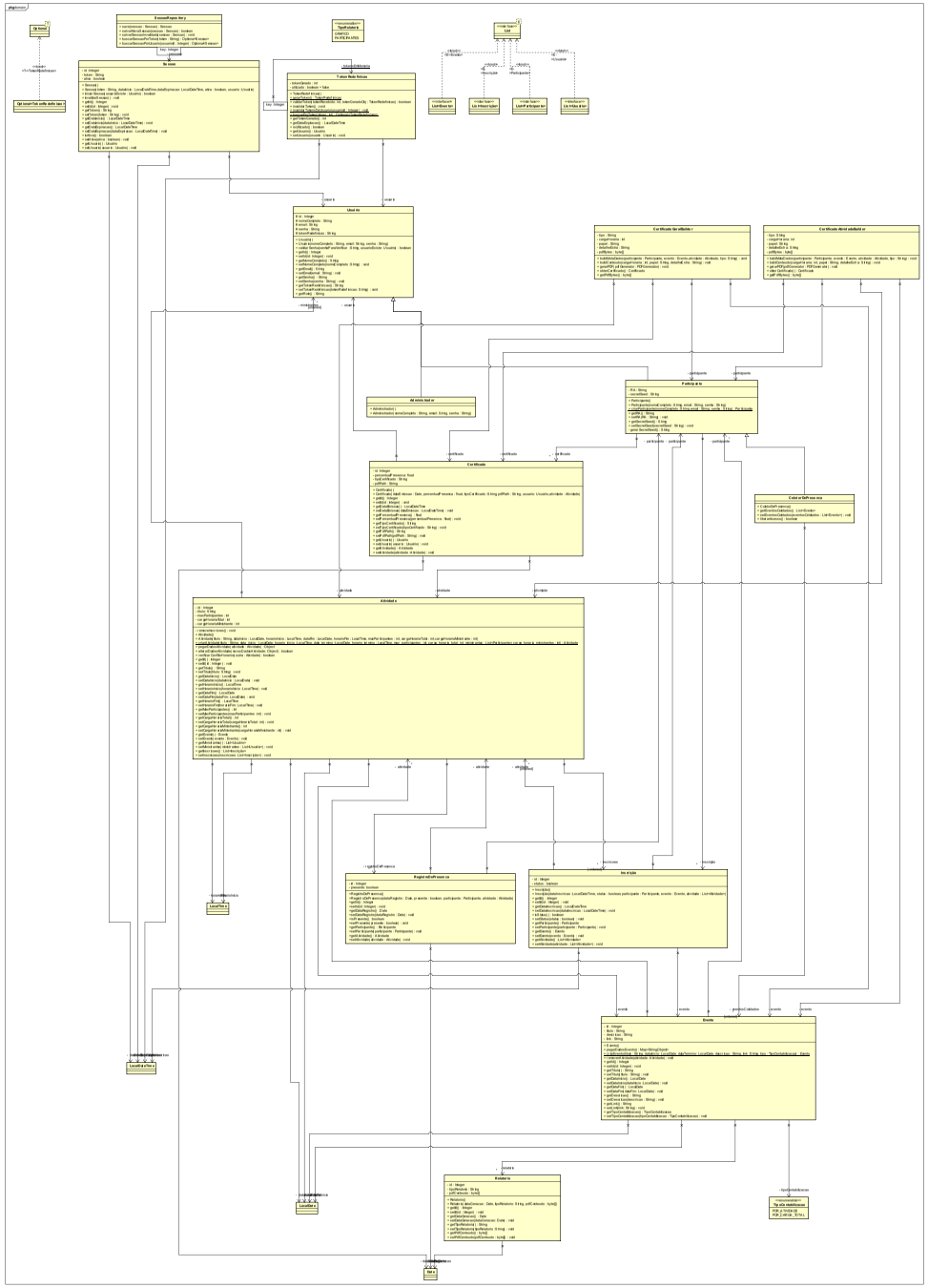
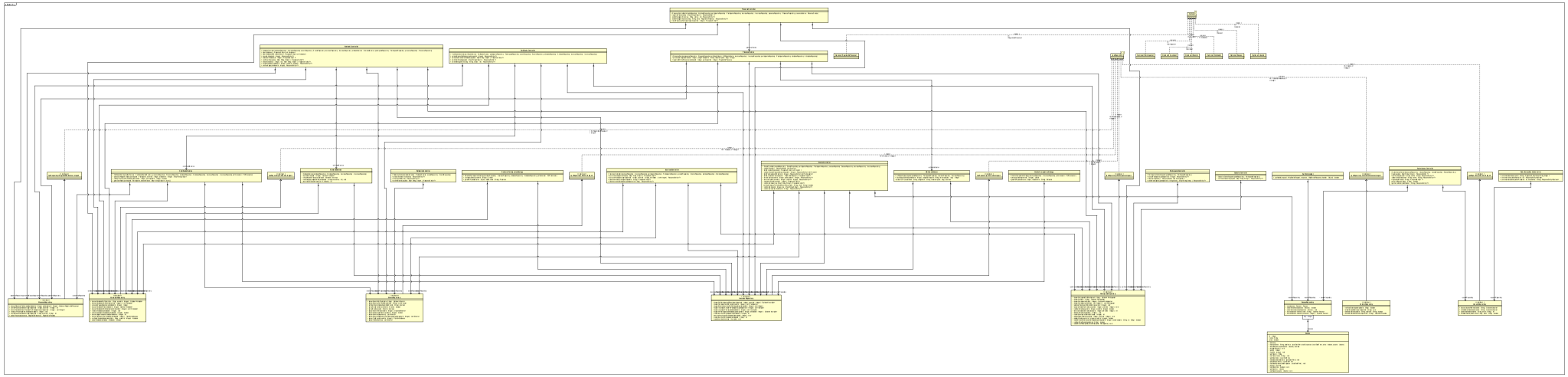
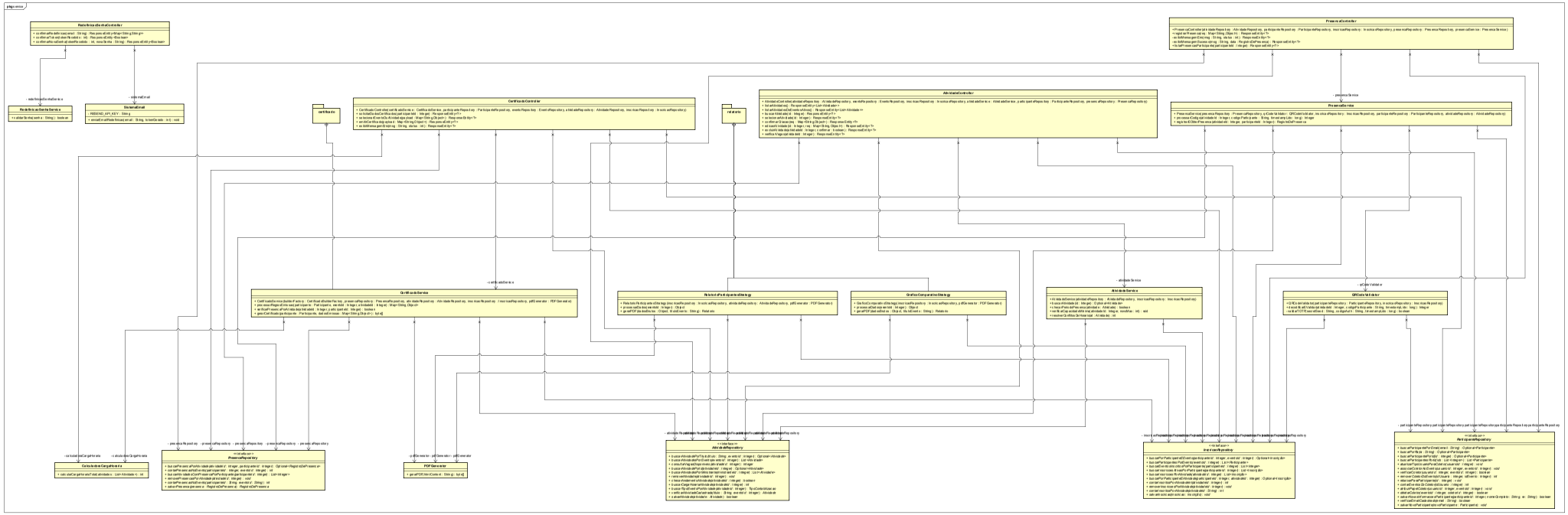

pkg evcomp
repository
controller
domain
config
service
EvcompApplication
+ main(args : String[]) : void
util

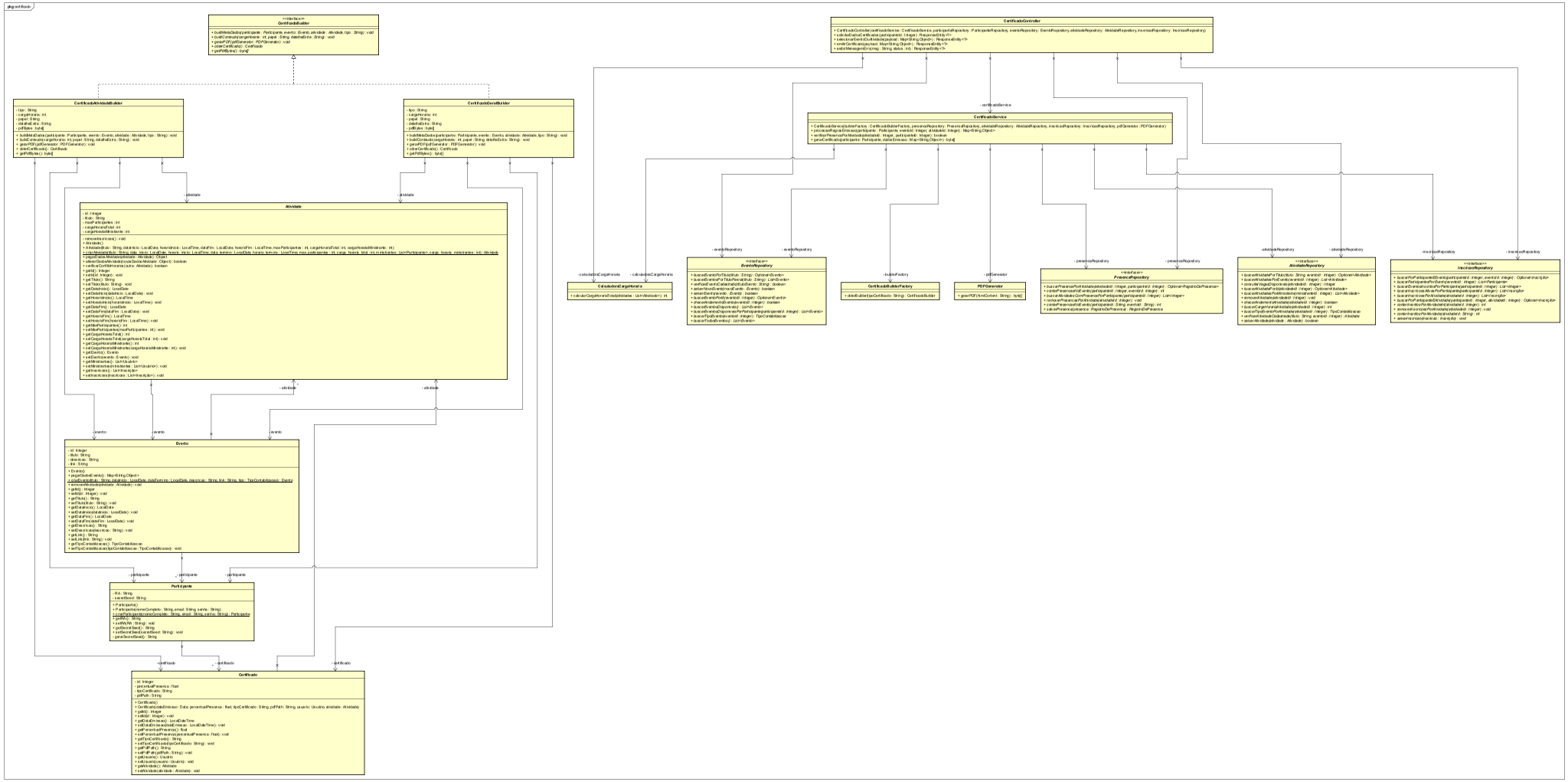

pkg config
<<interface>>
WebMvcConfigurer
<<interface>>
HandlerInterceptor
WebMvcConfig
+ addInterceptors(registry : InterceptorRegistry) : void
+ addCorsMappings(registry : CorsRegistry) : void
- authInterceptor
AuthInterceptor
+ preHandle(request : HttpServletRequest, response : HttpServletResponse, handler : Object) : boolean
- sessaoRepository
SessaoRepository
+ save(sessao : Sessao) : Sessao
+ salvarNovaSessao(sessao : Sessao) : boolean
+ salvarSessaoInvalidada(sessao : Sessao) : void
+ buscarSessaoPorToken(token : String) : Optional<Sessao>
+ buscarSessaoPorUsuario(usuarioId : Integer) : Optional<Sessao>

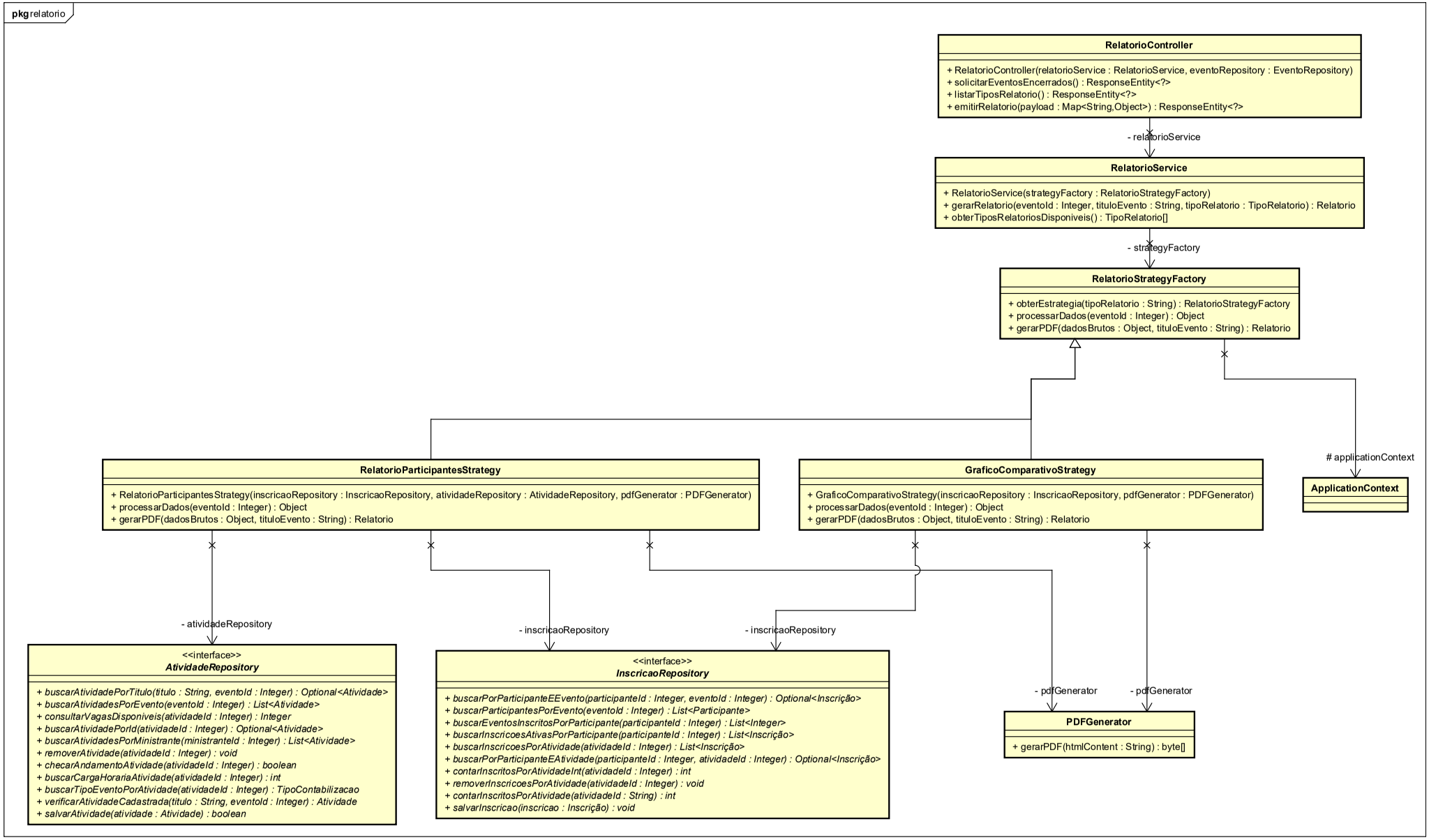


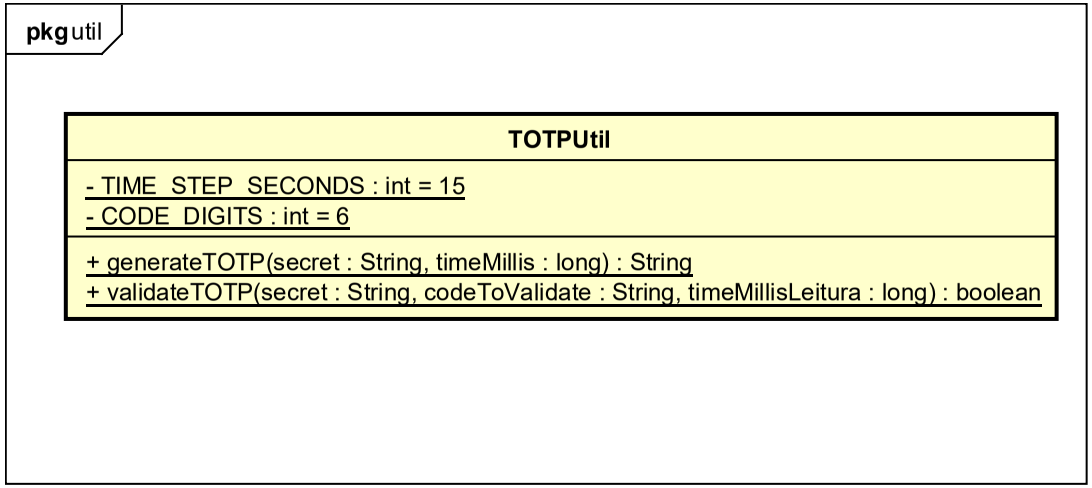

Up Home
Usuario
Usuario
Usuario




pkg[ca.uniroa]
RestaBrisaEstradaController
+ ca.uniroa.pelha.dado.cusaj.serial : String[]
+ ca.uniroa.ifal.web.stereo.file.cuida() : void
+ ca.uniroa.pelha.ualbert.ajt.stereo.file.cuida() : int
+ ca.uniroa.pelha.ualbert.ajt.stereo.file.cuida() : long
- ca.uniroa.pelha.dado.cusaj.serial
- ca.uniroa.ifal.web.stereo.file.cuida
RestaBrisaEstradaService
+ ca.uniroa.pelha.ualbert.ajt.stereo.file.cuida() : boolean
SendEmail
- RESEND_API_KEY : String
+ sendEmail(toEmail, bricsaLink, serial) : void
certificado
CertificadoController
+ Cert Ifcuida Controller(cert Ifc ability, ca.uniroa.pelha.ualbert.ajt.stereo.file.cuida, ca.uniroa.pelha.dado.cusaj.serial


pkg certificados
<<interface>>
CertificadoBuilder
+ buildMesAbierto(participante : Participante, evento : Evento, actividade : Atividade, tipo : String) : void
+ buildConsumo(cargaHoraria : int, papel : String, dataRefeitório : String) : void
+ getPDFGenerator() : PDFGenerator
+ getCertificado() : Certificado
+ getPdfBytes() : byte[]
CertificadoAtividadeBuilder
- tipo : String
- cargaHoraria : int
- papel : String
- dataRefeitório : String
- pdfBytes : byte[]
+ buildMesAbierto(participante : Participante, evento : Evento, actividade : Atividade, tipo : String) : void
+ buildConsumo(cargaHoraria : int, papel : String, dataRefeitório : String) : void
+ getPDFGenerator() : PDFGenerator
+ getCertificado() : Certificado
+ getPdfBytes() : byte[]
CertificadoEventoBuilder
- tipo : String
- cargaHoraria : int
- papel : String
- dataRefeitório : String
- pdfBytes : byte[]
+ buildMesAbierto(participante : Participante, evento : Evento, actividade : Atividade, tipo : String) : void
+ buildConsumo(cargaHoraria : int, papel : String, dataRefeitório : String) : void
+ getPDFGenerator() : PDFGenerator
+ getCertificado() : Certificado
+ getPdfBytes() : byte[]
- atividade
- atividade
Atividade
- id : Integer
- título : String
- maxParticipantes : int
- cargaHorariaFixa : int
- cargaHorariaMinistrante : int
- removeInscrições() : void
+ Atividade()
+ Atividade(título : String, descrição : LocalData, horaInício : LocalTime, dataFim : LocalData, horaFim : LocalTime, maxParticipantes : int, cargaHorariaFixa : int, cargaHorariaMinistrante : int)
+ criarAtividade(título : String, data_início : LocalDate, horário_início : LocalTime, data_término : LocalDate, horário_término : LocalTime, max_participantes : int, carga_horária_fixa : int, carga_horária_ministrantes : int) : Atividade
+ prepararDados(atividade : Atividade) : Object
+ podeAdicionarParticipante(participante : Participante, atividade : Object) : boolean
+ verificarConflitoHorario(outra : Atividade, atividade : Object) : boolean
+ getId() : Integer
+ setId(id : Integer) : void
+ getTítulo() : String
+ setTítulo(título : String) : void
+ getDataFim() : LocalDate
+ setDataFim(dataFim : LocalDate) : void
+ getHoraInício() : LocalTime
+ setHoraInício(horaInício : LocalTime) : void
+ getDataFim() : LocalDate
+ setDataFim(dataFim : LocalDate) : void
+ getHoraFim() : LocalTime
+ setHoraFim(horaFim : LocalTime) : void
+ getMaxParticipantes() : int
+ setMaxParticipantes(maxParticipantes : int) : void
+ getCargaHorariaTotal() : int
+ setCargaHorariaTotal(cargaHorariaTotal : int) : void
+ getCargaHorariaMinistrante() : int
+ setCargaHorariaMinistrante(cargaHorariaMinistrante : int) : void
+ getEvento() : Evento
+ setEvento(evento : Evento) : void
+ getMinistrantes() : List<Usuário>
+ setMinistrantes(ministrantes : List<Usuário>) : void
+ getInscrições() : List<Inscrição>
+ setInscrições(inscrições : List<Inscrição>) : void
- evento
- evento
- evento
Evento
- id : Integer
- título : String
- descrição : String
- link : String
+ Evento()
+ prepararDados(evento) : Map<String,Object>
+ criarEvento(título : String, descrição : LocalData, dataFim:Inicio : LocalDate, descrição : String, link : String, tipo : TipoContabilização) : Evento
+ removeAtividade(atividade : Atividade) : void
+ getId() : Integer
+ setId(id : Integer) : void
+ getTítulo() : String
+ setTítulo(título : String) : void
+ getDescrição() : LocalDate
+ setDescrição(descrição : LocalDate) : void
+ getDataFim() : LocalDate
+ setDataFim(dataFim : LocalDate) : void
+ getDescrição() : String
+ setDescrição(descrição : String) : void
+ getLink() : String
+ setLink(link : String) : void
+ getTipoContabilização() : TipoContabilização
+ setTipoContabilização(tipoContabilização : TipoContabilização) : void
- participante
- participante
- participante
Participante
- RA : String
- eventoEscol : String
+ Participante()
+ Participante(nomeCompleto : String, email : String, senha : String)
+ criarParticipante(nomeCompleto : String, email : String, senha : String) : Participante
+ getRA() : String
+ setRA(RA : String) : void
+ getEventoEscol() : String
+ setEventoEscol(eventoEscol : String) : void
+ gerarSenhaRandom() : String
- certificado
- certificado
- certificado
Certificado
- id : Integer
- percentualPresença : float
- tipoCertificado : String
- pdfPath : String
+ Certificado()
+ Certificado(dataEmissão : Data, percentualPresença : float, tipoCertificado : String, pdfPath : String, usuário : Usuário, atividade : Atividade)
+ getId() : Integer
+ setId(id : Integer) : void
+ getDataEmissão() : LocalDateTime
+ setDataEmissão(dataEmissão : LocalDateTime) : void
+ getPercentualPresença() : float
+ setPercentualPresença(percentualPresença : float) : void
+ getTipoCertificado() : String
+ setTipoCertificado(tipoCertificado : String) : void
+ getPdfPath() : String
+ setPdfPath(pdfPath : String) : void
+ getUsuário() : Usuário
+ setUsuário(usuário : Usuário) : void
+ getAtividade() : Atividade
+ setAtividade(atividade : Atividade) : void
CertificadoController
+ CertificadoController(certificadoService : CertificadoService, participantRepository : ParticipantesRepository, eventoRepository : EventoRepository, atividadeRepository : AtividadeRepository, inscricaoRepository : InscricaoRepository)
+ validarDadosCertificados(participanteId : Integer) : Response<String>
+ selecionarEventoDuvidosos(payload : Map<String,Object>) : ResponseEntity<?>
+ emitirCertificados(payload : Map<String,Object>) : ResponseEntity<?>
+ extrairMensagemErro(msg : String, status : int) : ResponseEntity<?>
- certificadoService
CertificadoService
+ CertificadoService(builderFactory : CertificadoBuilderFactory, presenceRepository : PresenceRepository, atividadeRepository : AtividadeRepository, inscricaoRepository : InscricaoRepository, pdfGenerator : PDFGenerator)
+ processeMegramEmissao(participante : Participante, eventoId : Integer, atividadeId : Integer) : Map<String,Object>
+ verificarPresenciaPorAtividade(atividadeId : Integer, participanteId : Integer) : boolean
+ gerarCertificado(participante : Participante, dataEmissao : Map<String,Object>) : byte[]
- calculadoraCargaHoraria
- calculadoraCargaHoraria
CalculadoraCargaHoraria
+ calcularCargaHorariaFixa(atividade : List<Atividade>) : int
- builderFactory
CertificadoBuilderFactory
+ criarBuilder(tipoCertificado : String) : CertificadoBuilder
- pdfGenerator
PDFGenerator
+ gerarPDF(htmlContent : String) : byte[]
- eventoRepository
- eventoRepository
<<interface>>
EventoRepository
+ buscarEventosPorTituloStatus(título : String) : Optional<Evento>
+ buscarEventosPorTituloPeriodo(título : String) : List<Evento>
+ verificarEventoComAtividadeAtivaEvento(evento : String) : boolean
+ salvarNovoEvento(novoEvento : Evento) : boolean
+ buscarEventosAtivos() : List<Evento>
+ buscarEventosPorEventoId(eventoId : Integer) : Optional<Evento>
+ checarAlcanceEventoEventoId(eventoId : Integer) : boolean
+ buscarEventosDisponiveis() : List<Evento>
+ buscarEventosDisponiveisPorParticipanteParticipanteId(participanteId : Integer) : List<Evento>
+ buscarTipoEventoEventoId(eventoId : Integer) : TipoContabilização
+ buscarTodosEventos() : List<Evento>
- presenceRepository
- presenceRepository
<<interface>>
PresenceRepository
+ buscarPresenceAtividadeAtividadeId(atividadeId : Integer, participanteId : Integer) : Optional<PresenceDePresença>
+ contarPresenceAtividadeEventoParticipanteId(eventoId : Integer, participanteId : Integer) : int
+ buscarAtividadesComPresenceNoParticipante(participanteId : Integer) : List<Integer>
+ removerPresenceAtividade(atividadeId : Integer) : void
+ contarPresenceAtividade(participanteId : String, eventoId : String) : int
+ salvarPresenceAgora(entra : PresenceDePresença) : PresenceDePresença
- atividadeRepository
- atividadeRepository
<<interface>>
AtividadeRepository
+ buscarAtividadePorTituloStatus(título : String, eventoId : Integer) : Optional<Atividade>
+ buscarAtividadesPorEventoEventoId(eventoId : Integer) : List<Atividade>
+ consultarVagasDisponiveisAtividade(atividadeId : Integer) : Integer
+ buscarAtividadePorId(atividadeId : Integer) : Optional<Atividade>
+ buscarAtividadesPorParticipanteParticipanteId(participanteId : Integer) : List<Atividade>
+ removerAtividade(atividadeId : Integer) : void
+ checarAlcanceAtividade(atividadeId : Integer) : boolean
+ buscarCargaHorariaAtividade(atividadeId : Integer) : int
+ buscarTipoEventoPorAtividade(atividadeId : Integer) : TipoContabilização
+ verificarAtividadeCadastradaStatus(título : String, eventoId : Integer) : Atividade
+ salvarAtividade(atividade : Atividade) : boolean
- inscricaoRepository
- inscricaoRepository
<<interface>>
InscricaoRepository
+ buscarPorParticipanteEEventoParticipanteId(participanteId : Integer, eventoId : Integer) : Optional<Inscrição>
+ buscarParticipantesPorEventoEventoId(eventoId : Integer) : List<Participante>
+ buscarEstatisticasInscricaoPorParticipanteParticipanteId(participanteId : Integer) : List<Integer>
+ buscarInscricoesAtivasPorParticipanteParticipanteId(participanteId : Integer) : List<Inscrição>
+ buscarInscricoesPorAtividadeAtividadeId(atividadeId : Integer) : List<Inscrição>
+ buscarParticipantesEEstatisticasParticipanteId(participanteId : Integer, atividadeId : Integer) : Optional<Inscrição>
+ existeInscricaoAtivaEmAtividade(atividadeId : Integer) : boolean
+ removerInscricaoPorAtividade(atividadeId : Integer) : void
+ contarInscricoesPorAtividade(atividadeId : Integer) : int
+ salvarInscricao(inscricao : Inscrição) : void


pkg relatorio
RelatorioController
+ RelatorioController(relatorioService : RelatorioService, eventoRepository : EventoRepository)
+ solicitarEventosEncerrados() : ResponseEntity<?>
+ listarTiposRelatorio() : ResponseEntity<?>
+ emitirRelatorio(payload : Map<String,Object>) : ResponseEntity<?>
- relatorioService
RelatorioService
+ RelatorioService(strategyFactory : RelatorioStrategyFactory)
+ gerarRelatorio(eventoId : Integer, tituloEvento : String, tipoRelatorio : TipoRelatorio) : Relatorio
+ obterTiposRelatoriosDisponiveis() : TipoRelatorio[]
- strategyFactory
RelatorioStrategyFactory
+ obterEstrategia(tipoRelatorio : String) : RelatorioStrategyFactory
+ processarDados(eventoId : Integer) : Object
+ gerarPDF(dadosBrutos : Object, tituloEvento : String) : Relatorio
# applicationContext
ApplicationContext
RelatorioParticipantesStrategy
+ RelatorioParticipantesStrategy(inscricaoRepository : InscricaoRepository, atividadeRepository : AtividadeRepository, pdfGenerator : PDFGenerator)
+ processarDados(eventoId : Integer) : Object
+ gerarPDF(dadosBrutos : Object, tituloEvento : String) : Relatorio
GraficoComparativoStrategy
+ GraficoComparativoStrategy(inscricaoRepository : InscricaoRepository, pdfGenerator : PDFGenerator)
+ processarDados(eventoId : Integer) : Object
+ gerarPDF(dadosBrutos : Object, tituloEvento : String) : Relatorio
- atividadeRepository
- inscricaoRepository
- inscricaoRepository
- pdfGenerator
- pdfGenerator
<<interface>>
AtividadeRepository
+ buscarAtividadePorTitulo(titulo : String, eventoId : Integer) : Optional<Atividade>
+ buscarAtividadesPorEvento(eventoId : Integer) : List<Atividade>
+ consultarVagasDisponiveis(atividadeId : Integer) : Integer
+ buscarAtividadePorId(atividadeId : Integer) : Optional<Atividade>
+ buscarAtividadesPorMinistrante(ministranteId : Integer) : List<Atividade>
+ removerAtividade(atividadeId : Integer) : void
+ checarAndamentoAtividade(atividadeId : Integer) : boolean
+ buscarCargaHorariaAtividade(atividadeId : Integer) : int
+ buscarTipoEventoPorAtividade(atividadeId : Integer) : TipoContabilizacao
+ verificarAtividadeCadastrada(titulo : String, eventoId : Integer) : Atividade
+ salvarAtividade(atividade : Atividade) : boolean
<<interface>>
InscricaoRepository
+ buscarPorParticipanteEEvento(participanteId : Integer, eventoId : Integer) : Optional<Inscrição>
+ buscarParticipantesPorEvento(eventoId : Integer) : List<Participante>
+ buscarEventosInscritosPorParticipante(participanteId : Integer) : List<Integer>
+ buscarInscricoesAtivasPorParticipante(participanteId : Integer) : List<Inscrição>
+ buscarInscricoesPorAtividade(atividadeId : Integer) : List<Inscrição>
+ buscarPorParticipanteEAtividade(participanteId : Integer, atividadeId : Integer) : Optional<Inscrição>
+ contarInscritosPorAtividadeInt(atividadeId : Integer) : int
+ removerInscricoesPorAtividade(atividadeId : Integer) : void
+ contarInscritosPorAtividade(atividadeId : String) : int
+ salvarInscricao(inscricao : Inscrição) : void
PDFGenerator
+ gerarPDF(htmlContent : String) : byte[]


pkg util
TOTPUtil
- TIME_STEP_SECONDS : int = 15
- CODE_DIGITS : int = 6
+ generateTOTP(secret : String, timeMillis : long) : String
+ validateTOTP(secret : String, codeToValidate : String, timeMillisLeitura : long) : boolean